

up image. It's more typical (and strongly suggested) to pass in paths for the optional images, like so:

```
restartButton = pygamewidgets.CustomButton(window, (100, 430),
                                           'images/RestartButtonUp.png',
                                           down='images/RestartButtonDown.png',
                                           over='images/RestartButtonOver.png',
                                           disabled='images/RestartButtonDisabled.png',
                                           soundOnClick='sounds/blip.wav',
                                           nickname='restart')
```

Here we also specified a sound effect that should be played when the user clicks the button, and we provided an internal nickname we can use later.

Using Buttons

After instantiation, here's some typical code to use a button object, `oButton`, independent of it being a `TextButton` or a `CustomButton`:

```
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

        if oButton.handleEvent(event):
            # User has clicked this button
            <Any code you want to run here when the button is clicked>

--- snip ---
oButton.draw() # at the bottom of the while loop, tell it to draw
```

Every time we detect an event, we need to call the `handleEvent()` method of the button to allow it to react to the user's actions. This call normally returns `False` but will return `True` when the user completes a click on the button. At the bottom of the main `while` loop, we need to call the `draw()` method of the button to allow it to draw itself.

Text Output and Input

As we saw in Chapter 6, handling text input and output in pygame is tricky, but here I'll introduce new classes for a text display field and an input text